Introduction to LangGraph

Monday, March 23, 2026 6:02 PM

Recap:

✓Youtube → Information

↓ link ✓

Document loader → Youtube loader

↓

Transcript

↓

Runnable Brach (if)

1000-5000

According

Small

<1000 words

↓

Sys & Human ↔ LLM.

↓

Article (Text)

↓

LLM → Create a webpage — JS, html, CSS

↓

Article- webpage

Project - 3.

↓

Youtube - vid- link

↓

Applcn

↓

webpage article

#1 Token

Trans > 1000 (5000).

agent - Lang chain

↓

Middleware summarizer

Chunks → add Chunks recursively.

↓

article

↓

LLM — JS html CSS

↓

Article webpage

GenAI → prompt Engs.

Lang chain → Interact LLM → prompt → Memory → Runnables →

LCEL-Chain - agent → Tool Calling

↓

Multi agent

RAG → Indexing Ret Gen

Doc load, Splitter, emb, vector stores → Retrival → Generation

Chromadb

LangGraph → why?

↓

what?

lib → Tools (func | class)

↓

Build applcn powered by LLMs

In comp control exercise with langgraph ✓ Complex work flows ✓ } In comp with other frameworks.

✓ & greater ✓ control ✓

popular Tools

- Crew AI ×
- Autogen ×
- open AI SDK ×
- Google Gemini ADK ×

LangChain

Chain

LCEL chain

↓

PT | LLM | parser → all the elements of chain are arranged in a sequence

↓

Branching

Through LCEL we cannot build very complex int b/w elements } But we cannot any element in chain to any other element in chain

In lang chain → every applcn → LCEL chain

$e_1 \mid e_2 \mid e_3 \mid e_4 \mid e_5 \mid \ldots$

↓

e_1 | e_2 | e_3 | e_4 | e_5

↓

Runnable

In lang Graph → every appln is a graph

Chatbot

START

↓

Chatbot

↓

End

Langchain

pT | LLM | parser

Bec every appln is a graph

↓ easily understand | interpret

easily Debug

But → lang Graph

→ Steep Learning Curve

Agenda

Build a graph

Build a chatbot with options in Lang Graph

RAG Agent

Multi Agent

GenerAI

↓

Lang Graph

↓

Hospital admin help

All the Concepts

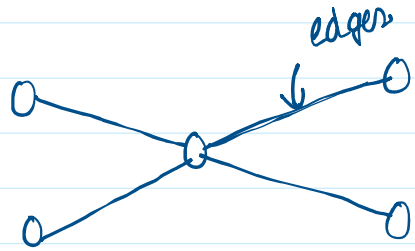
Lang graph

↓

Nodes

edges

START



○ Nodes

↓

perform some operations

Struct of appln

First Node START

→ perform operation

First Node START

Logic ⇒ Comb Nodes & edges. → perform operation

Last Node END

Through out the graph ⇒ every node accesses ⇒ State

Persistent | Persistent memory.

State → obj → Mostly Dictionary

Key: val pairs

State

Start

Logic

End

State will be updates

Nodes will update state

State ← Modified / updated state